

Image Processing Tutorial

Spatial Filtering & Histogram Processing

(Sheet 4 / Tutorial 4) • ECE 359

Adnan Zahran

Faculty of Engineering • Ain Shams University
Communications Systems Engineering Program

Spring 2026

Outline

- 1 1-D Spatial Filtering
- 2 2-D Spatial Filtering
- 3 Basic Filter Computations
- 4 Correlation vs Convolution
- 5 First vs Second Derivatives
- 6 Image Padding Effects
- 7 Combined Enhancement Operations
- 8 Textbook Problems

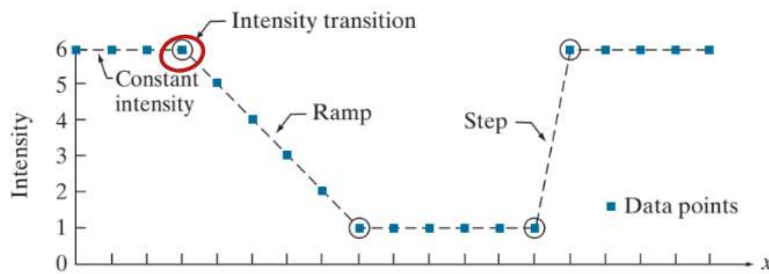
1D Laplacian (Derivative operator)

First order derivative

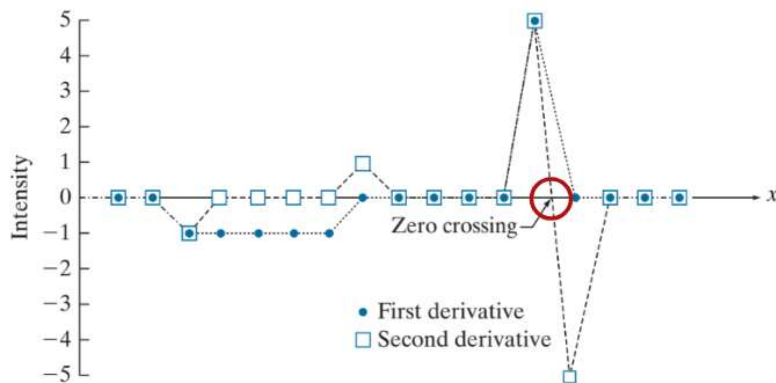
$$\frac{\partial f}{\partial x} = f(x+1) - f(x)$$

Second order derivative

$$\frac{\partial^2 f}{\partial x^2} = f(x+1) + f(x-1) - 2f(x)$$



Values of scan line	6	6	6	6	5	4	3	2	1	1	1	1	1	1	6	6	6	6	6
1st derivative	0	0	-1	-1	-1	-1	-1	0	0	0	0	0	0	5	0	0	0	0	0
2nd derivative	0	0	-1	0	0	0	0	1	0	0	0	0	0	5	-5	0	0	0	0



2D Laplacian

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

$$\frac{\partial^2 f}{\partial x^2} = f(x+1, y) + f(x-1, y) - 2f(x, y)$$

and, similarly, in the y-direction, we have

$$\frac{\partial^2 f}{\partial y^2} = f(x, y+1) + f(x, y-1) - 2f(x, y)$$

$$\nabla^2 f(x, y) = f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y)$$

0	1	0
1	-4	1
0	1	0

1	1	1
1	-8	1
1	1	1

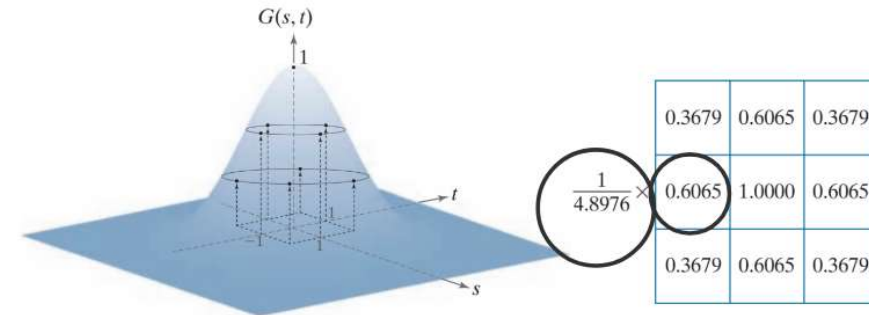
$$g(x, y) = f(x, y) + c [\nabla^2 f(x, y)]$$

$c = -1$ for negative center.
+1 positive

a b

FIGURE 3.35

(a) Sampling a Gaussian function to obtain a discrete Gaussian kernel. The values shown are for $K = 1$ and $\sigma = 1$. (b) Resulting 3×3 kernel [this is the same as Fig. 3.31(b)].



$$w(s, t) = G(s, t) = Ke^{-\frac{s^2 + t^2}{2\sigma^2}}$$

★ Why kernels with odd sizes are used?
unique center

unsharp & High boost Filters

Subtracting an unsharp (smoothed) version of an image from the original image is process that has been used since the 1930s by the printing and publishing industry to sharpen images. This process, called *unsharp masking*, consists of the following steps:

1. Blur the original image.
2. Subtract the blurred image from the original (the resulting difference is called the *mask*.)
3. Add the mask to the original.

Letting $\bar{f}(x, y)$ denote the blurred image, the mask in equation form is given by:

$$g_{\text{mask}}(x, y) = f(x, y) - \bar{f}(x, y) \quad (3-55)$$

Then we add a weighted portion of the mask back to the original image:

$$g(x, y) = f(x, y) + k g_{\text{mask}}(x, y) \quad (3-56)$$

K $\left\{ \begin{array}{ll} K < 1 & \text{reduces effect} \\ K = 1 & \text{unsharp} \\ K > 1 & \text{High boost} \end{array} \right.$

Original signal

Blurred signal

Unsharp mask

Sharpened signal

Unsharp Masking

Steps:

- Blur the original: $\bar{f} = \text{blur}(f)$
- Extract mask: $g_{\text{mask}} = f - \bar{f}$ (high-frequency details)
- Add back: $g = f + k \cdot g_{\text{mask}}$

Formula:

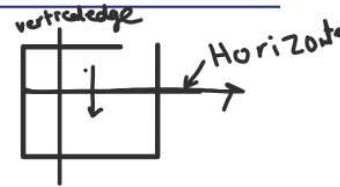
$$g(x, y) = f(x, y) + k(f(x, y) - \bar{f}(x, y))$$

Parameter: $k \in [0.5, 2.5]$

- $k = 1$: Standard unsharp masking
- $k > 1$: Highboost filtering (stronger sharpening)

Gradient operator (Sobel & Roberts)

$$\nabla f \equiv \text{grad}(f) = \begin{bmatrix} g_x \\ g_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$



Roberts Kernel $G_x = \begin{bmatrix} +1 & 0 \\ 0 & -1 \end{bmatrix}$ $G_y = \begin{bmatrix} 0 & +1 \\ -1 & 0 \end{bmatrix}$ diagonal edges
 $G_x > 0$ Intensity increase from $\ominus \rightarrow \oplus$

Sobel Correlation Kernels

Horizontal Gradient g_x

$$g_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Detects: Vertical edges (Horizontal change)
 (intensity changes left-to-right)

Vertical Gradient g_y

$$g_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Detects: Horizontal edges (Vertical change)
 (intensity changes top-to-bottom)

Gradient Magnitude

Exact: $M(x, y) = \sqrt{g_x^2 + g_y^2}$

Approximation (faster): $M(x, y) = |g_x| + |g_y|$

} for both $M = \text{how strong is edge?}$

1D Sobel Scan

$$M = G = \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 10 & \boxed{10} & 80 \end{bmatrix} \\ -10 + 80 = 70 \\ \begin{bmatrix} 10 & \boxed{70} & 80 \end{bmatrix}$$

$G_x > 0$ Left $\rightarrow$ right Intensity increase
 $G_y > 0$ up $\rightarrow$ down increase
 opposite directions viceversa

Gradient vs Laplacian

Gradient (1st derivative):

- + Produces thick edges with **magnitude and direction**
- Requires thresholding + non-maximum suppression for thin edges

Laplacian (2nd derivative):

- + Produces **zero-crossings** $\rightarrow$ inherently thin edges
- Very noise-sensitive, no directional information

Computational Optimization: Separability

FYI

A 2-D kernel $H(s, t)$ is **separable** if it can be written as:

$$H(s, t) = h_{\text{col}}(s) \cdot h_{\text{row}}(t)$$

i.e., a product of a column vector and a row vector.

How to Check for Separability

$$H = \mathbf{v} \cdot \mathbf{u}^T$$

Verify by checking $\text{rank}(H) = 1$ (the matrix has rank 1).

Examples

✓ Gaussian is separable:

$$H_G = \frac{1}{16} \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} [1 \ 2 \ 1] = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

✓ Box filter is separable:

$$H_{\text{box}} = \frac{1}{9} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} [1 \ 1 \ 1]$$

✗ Laplacian is NOT separable (rank > 1).

Computational Benefit

- **Standard 2-D convolution:** $O(n^2)$ multiplications per pixel
- **Separable filter:** $O(2n)$ multiplications per pixel (1-D horizontal + 1-D vertical)
- For $n = 5$: 25 ops $\rightarrow$ 10 ops = **2.5× speedup**
- For $n = 15$: 225 ops $\rightarrow$ 30 ops = **7.5× speedup**

Worked Example: Separable 3×3 Gaussian

$$H_G = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} = \frac{1}{4} \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \otimes \frac{1}{4} [1 \ 2 \ 1]$$

Step 1: Apply $h_r = \frac{1}{4} [1, 2, 1]$ **horizontally** to each row

For row $y = 2$: $[104, 48, 108, 109, 110]$

- $f'[0, 2] = \frac{1}{4} (0 \cdot 1 + 104 \cdot 2 + 48 \cdot 1) = \frac{256}{4} = 64$ (zero-padded left)
- $f'[1, 2] = \frac{1}{4} (104 \cdot 1 + 48 \cdot 2 + 108 \cdot 1) = \frac{308}{4} = 77$
- $f'[2, 2] = \frac{1}{4} (48 \cdot 1 + 108 \cdot 2 + 109 \cdot 1) = \frac{373}{4} = 93.25$

Step 2: Apply $h_c = \frac{1}{4} [1, 2, 1]^T$ **vertically** to each column of $f' \rightarrow$ final output g .

Result equals the direct 2-D convolution ✓

📋 Pick Your Filter in 3 Questions

1. What kind of noise?

- Gaussian noise $\rightarrow$ **Gaussian Filter**
- Salt-and-pepper $\rightarrow$ **Median Filter**
- Only pepper (dark dots) $\rightarrow$ **Max Filter**
- Only salt (bright dots) $\rightarrow$ **Min Filter**

2. Do you want to sharpen or detect edges?

- Smooth first $\rightarrow$ then **Laplacian** (fine detail) or **Sobel** (gradient edges)
- Need tunable sharpening $\rightarrow$ **High-Boost**

3. Do you need a feature map, not a clean image?

- Gradient magnitude $\rightarrow$ **Sobel**

Filters comparison

Filter	Type	Purpose	Use When	Edges	Handles
Box (Average)	Linear	Fast smoothing	Speed matters, quality doesn't	Strongly blurred	Mild random noise
Gaussian	Linear	High-quality smoothing	Pre-edge-detection	Smooth, controlled	Gaussian noise
Laplacian	Linear	Edge/detail boost	After smoothing	Enhanced (noise-sensitive)	Details post-denoising
Sobel	Linear	Gradient edge detection	Magnitude + direction	Thick, robust	After mild smoothing
High-Boost	Linear	Tunable sharpening	Low-contrast enhancement	Strong, tunable	Post-smoothing HF
Median	Nonlinear	Impulse denoising	Salt-and-pepper + edges	Preserves edges	Salt-and-pepper
Max	Nonlinear	Remove dark noise	Pepper removal, dilation	Expands bright	Dark (pepper)
Min	Nonlinear	Remove bright noise	Salt removal, dilation	Expands dark	Bright (salt)

Laplacian Sharpening (Problem 9)

Given: A 3×3 image region:

Image Region

$$\begin{bmatrix} 5 & 5 & 5 \\ 5 & 10 & 5 \\ 5 & 5 & 5 \end{bmatrix}$$

Tasks:

- 1 Apply basic Laplacian mask (center = -4)
- 2 Apply Laplacian with diagonals (center = -8)
- 3 Compute sharpened output: $g(x, y) = f(x, y) - \nabla^2 f$

Solution — Problem 9

Problem 9 Solution

Laplacian mask

$$H_L = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad \text{4 neighbor Laplace}$$

$$H_{L8} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad \text{8 neighbor}$$

Summation equals zero
($1 \times 8 - 8$) = 0

$$\begin{bmatrix} 5 & 5 & 5 \\ 5 & 10 & 5 \\ 5 & 5 & 5 \end{bmatrix} \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix} = +20$$

$g = 10 + 20 =$

pixel (10)

$$\nabla^2 f = \begin{bmatrix} 5 & 5 & 5 \\ 5 & 10 & 5 \\ 5 & 5 & 5 \end{bmatrix} \cdot \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} = 10 \times -4 + 5 \times 4 = -20$$

$$g(x,y) = f - \nabla^2 f = 10 - (-20) = 30$$

$$\text{New image} = \begin{bmatrix} 5 & 5 & 5 \\ 5 & 30 & 5 \\ 5 & 5 & 5 \end{bmatrix}$$

$$\begin{bmatrix} 5 & 5 & 5 \\ 5 & 10 & 5 \\ 5 & 5 & 5 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix} = 5 \times 8 - 10 \times 8 = -40$$

$$g = 10 - (-40) = 50$$

$$\begin{bmatrix} 5 & 5 & 5 \\ 5 & 50 & 5 \\ 5 & 5 & 5 \end{bmatrix}$$

Sobel Gradient Magnitude (Problem 10)

Given: A 3×3 image region:

Image Region

$$\begin{bmatrix} 10 & 10 & 10 \\ 10 & 50 & 10 \\ 10 & 10 & 10 \end{bmatrix}$$

Use: Sobel operators

$$\begin{array}{l} \text{Vertical edges} \\ g_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \end{array} \quad \begin{array}{l} \text{Horizontal edges} \\ g_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \end{array}$$

Find: $M(x, y) = |g_x| + |g_y|$ at center pixel
approximation!

$$\begin{aligned} |\nabla f| &= (G_x^2 + G_y^2)^{1/2} \\ g &= f + |\nabla f| \end{aligned}$$

Solution — Problem 10

Problem 10 Solution

$$\begin{bmatrix} 10 & 10 & 10 \\ 10 & 50 & 10 \\ 10 & 10 & 10 \end{bmatrix} \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} = -(10+20+10) + (10+20+10) = 0$$

$$\begin{bmatrix} 10 & 10 & 10 \\ 10 & 50 & 10 \\ 10 & 10 & 10 \end{bmatrix} \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} = -(10+20+10) + (10+20+10) = 0$$

$$M \approx |G_x| + |G_y| = 0 \text{ approximate}$$

$$= \sqrt{|G_x|^2 + |G_y|^2} = \text{exact}$$

$$\begin{bmatrix} 10 & 10 & 10 \\ 10 & 0 & 10 \\ 10 & 10 & 10 \end{bmatrix}$$

Circular symmetric around center pixel

Unsharp Masking (Problem 11)

Given:

- Original pixel value: $f(x, y) = \underline{100}$
- Blurred value: $\bar{f}(x, y) = \underline{80}$

Find sharpened output using:

Formulas

$$g_{mask}(x, y) = f(x, y) - \bar{f}(x, y)$$

$$g(x, y) = f(x, y) + \textcircled{k} g_{mask}(x, y)$$

- (a) Unsharp masking with $k = 1$
- (b) Highboost filtering with $k = 2.5$
- (c) Compare the values

Solution — Problem 11

Problem 11 Solution

$$g_{\text{mask}} = 100 - 80 = 20$$

$$g_{(xy)} = 100 + K(20) = 120 \text{ unsharp}$$

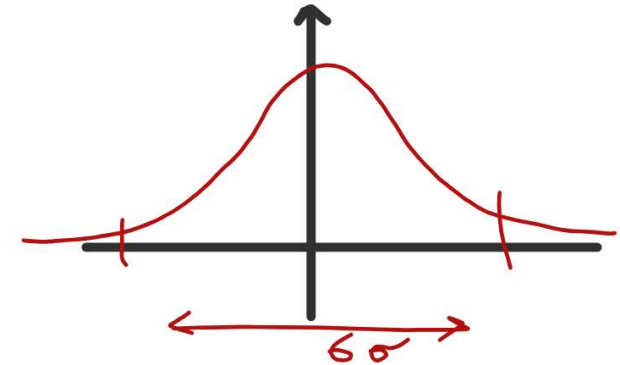
$$100 + 50 = 150 \text{ High boost}$$

Gaussian Kernel Size (Problem 12)

Given: Standard deviation $\sigma = 5$

Tasks:

- 1 Determine minimum kernel size using: size $\geq 6\sigma$
- 2 What is the smallest odd kernel size? $30 \rightarrow 31 \times 31$
- 3 Why do we use odd-sized kernels?
- 4 What happens if we use a larger kernel (e.g., 85×85)?



Concept: The 6σ rule ensures we capture 99.7% of Gaussian distribution

④ 31×31 already covered necessary values
The Larger kernel is more expensive

Correlation vs Convolution (Problem 13)

Given: A filter kernel:

Filter w

$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

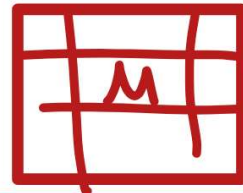
Tasks:

- 1 Perform correlation with image region
- 2 Perform convolution (rotate filter 180°)
- 3 Compare the two results *w antisymmetric*
- 4 When are correlation and convolution equivalent?

→ *Centro-symmetric filters*
→ *Gaussian, box filter, Laplacian*

assume this is the image portion

$$A = \begin{bmatrix} 105 & 107 & 108 \\ 48 & 108 & 109 \\ 110 & 104 & 106 \end{bmatrix}$$



Solution — Problem 13

Problem 13 Solution

a)
$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} \begin{bmatrix} 10^5 & 10^7 & 10^8 \\ 48 & 10^8 & 10^9 \\ 110 & 10^4 & 10^6 \end{bmatrix} = (10^5 + 10^8 + 10^7 \times 2) - (110 + 10^6 + 2 \times 10^4)$$
$$= +1$$

b)
$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \quad // \quad = -1$$

c) Kernel is antisymmetric

d)

First vs Second Derivatives (Problem 14)

Given: Intensity profile:

[5, 5, 5, 8, 12, 15, 15, 15]

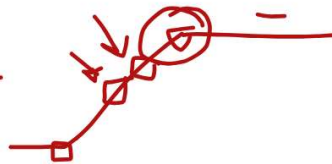
Compute:

(a) First derivative: $\frac{\partial f}{\partial x} = \underline{f(x+1)} - \underline{f(x)}$

(b) Second derivative: $\frac{\partial^2 f}{\partial x^2} = \underline{f(x+1)} + \underline{f(x-1)} - \underline{2f(x)}$

Analyze:

- Response in constant regions
- Response at start of ramp
- Response along ramp
- Response at end of ramp



Solution — Problem 14

Problem 14 Solution

x	0	1	2	3	4	5	6	7
$f(x)$	5	5	5	8	12	15	15	15
$f'(x)$	0	0	+3	+4	+3	0	0	0
$f''(x)$	0	0	+3	1	-1	-3	0	-

→ Replicate padding

) from 1st

$$f''(x) = f(x+1) + f(x-1) - 2f(x)$$

$$f''(1) = f(2) + f(0) - 2f(1) = 5 + 5 - 2(5) = 0$$

$$f''(2) = f(3) + f(1) - 2f(2) = 8 + 5 - 10 = 3$$

Zero crossing of f'' marks
edge location

Image Padding Effects (Problem 15)

Given: 5×5 image, apply 3×3 averaging filter

Compare three padding methods:

- ① **Zero padding:** Add zeros around border
- ② **Replicate padding:** Repeat edge pixels
- ③ ~~**Mirror padding:** Reflect image across border~~

Tasks:

- Compute filtered output for each method
- Observe border effects
- Which method produces best results and why?

Solution — Problem 15

Problem 15 Solution

Method	Border Effect	Quality
Zero padding	Missing neighbors = 0 → darkens borders	✗ <u>Artificial dark halo</u>
Replicate	Extends edge pixel value outward	✓ Better, slight plateau
Mirror	Reflects image across border	✓ Best — no discontinuity

Best method: Mirror padding — creates the smoothest transition and avoids artificial edges.

Combined Enhancement Operations (Problem 16)

Scenario: Enhance bone scan image using combination of techniques Matlab & theory

Steps:

- ① Apply Laplacian: $f_L = \nabla^2 f$
- ② Sharpen: $f_1 = f + c \cdot f_L$
- ③ Compute Sobel gradient: $f_2 = \text{sobel}(f)$
- ④ Smooth gradient: $f_3 = \text{blur}(f_2)$
- ⑤ Create mask: $f_4 = f_1 \times f_3$
- ⑥ Final result: $g = f + f_4$

$$f_L = \nabla^2 f \Rightarrow f_1 = f + c \nabla^2 f$$
$$f_2 = |\nabla f|, \quad f_3 = \text{blur}(|\nabla f|), \quad f_4 = f_1 \cdot f_3$$

$$g(x, y) = f(x, y) + [f(x, y) + c \nabla^2 f(x, y)] \cdot \text{blur}(|\nabla f(x, y)|)$$

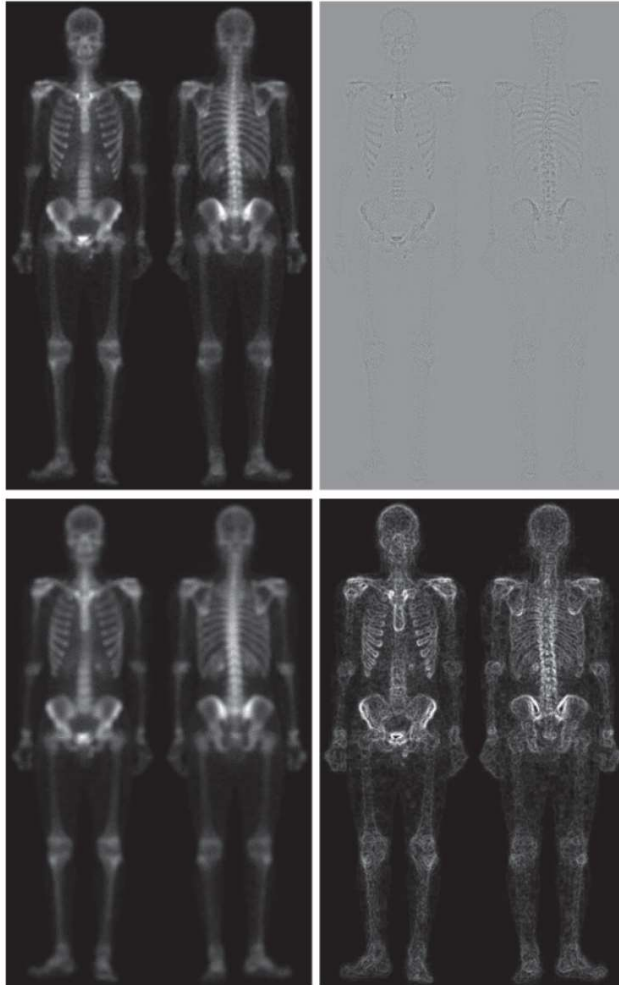
Derive: Final equation $g(x, y)$ in terms of original $f(x, y)$ ↗ Mathematical derivation

Solution — Problem 16

a b
c d

FIGURE 3.57

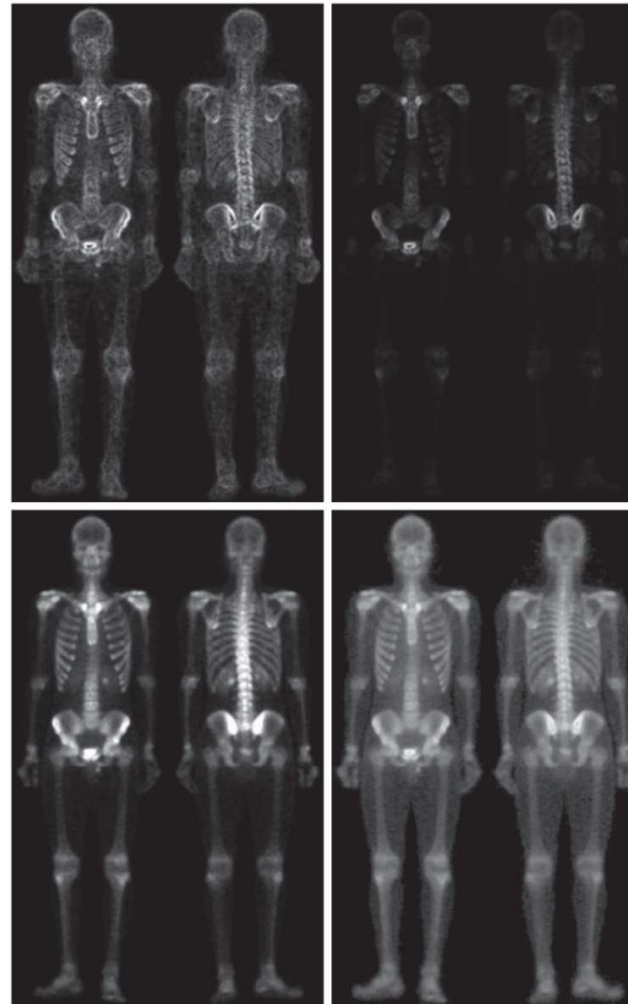
(a) Image of whole body bone scan.
(b) Laplacian of (a).
(c) Sharpened image obtained by adding (a) and (b).
(d) Sobel gradient of image (a). (Original image courtesy of G.E. Medical Systems.)



e f
g h

FIGURE 3.57

(Continued)
(e) Sobel image smoothed with a 5×5 box filter.
(f) Mask image formed by the product of (b) and (e).
(g) Sharpened image obtained by the adding images (a) and (f).
(h) Final result obtained by applying a power-law transformation to (g). Compare images (g) and (h) with (a). (Original image courtesy of G.E. Medical Systems.)



Order of Operations: Averaging then Laplacian (Problem 17)

Problem 17 (G&W 3.23)

In an application, an **averaging mask** is first applied to reduce noise, then a **Laplacian mask** is applied to enhance small details.

Would the result be the same if the order were reversed?

Justify your answer using properties of linear operations.

Recall

Both averaging and Laplacian are **linear** spatial filters.

For linear filters: $F_1 * (F_2 * I) = F_2 * (F_1 * I)$ (convolution is commutative).

Solution — Problem 17

✓ Answer

Yes — the result is identical regardless of order.

$$A * (L * I) = (A * L) * I = (L * A) * I = L * (A * I)$$

Convolution is **associative and commutative** for linear filters.

🔥 Practical Note

Applying averaging **first** (noise reduction) then Laplacian (sharpening) is preferred in practice — avoids amplifying noise with the Laplacian.

Averaging Mask Size Effects (Problem 18)

Problem 18 (G&W 3.21)

Three images were blurred with **square averaging masks** of sizes $n = 23$, 25, 45 respectively. The vertical bars in (a) and (c) are blurred but clearly separated, while the bars in (b) have **merged** — despite (b) using a *smaller* mask than (c).

Explain why.

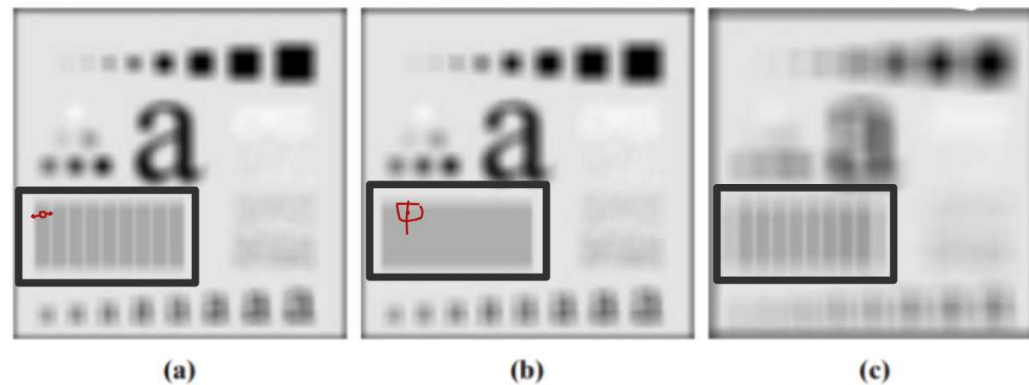


Image (a) — mask $n = 23$

Image (b) — mask $n = 25$, bars merged

Image (c) — mask $n = 45$, bars separated

Solution — Problem 18

Problem 18 Solution

Image	Mask	Result	Reason
(a)	$n = 23$	Bars separated	Mask < bar period; never covers two bars simultaneously
(b)	$n = 25$	Bars merged	Mask = bar period $\rightarrow$ constant average at every position
(c)	$n = 45$	Bars separated	Mask is large but $\neq$ period $\rightarrow$ average varies with position

Rule: Merging occurs when $n = \text{bar period}$ — the box filter becomes a period-matched integrator.



Identifying the Smoothing Filter Used (Problem 19)

Problem 19

The left image was filtered to produce the right image. The **small black square** (lower-right of original) shows the mask size. The filter was one of: **(1) Averaging**, **(2) Ideal Low-Pass**, **(3) Gaussian Low-Pass**, **(4) Median**.

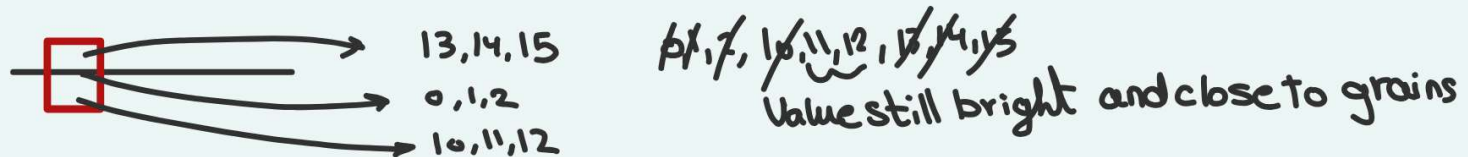
For each filter, give **at least one reason** why it was or was not the filter used.



Solution — Problem 19

Problem 19 Solution

- 1) Averaging Filters box → influence output by far just as close pixels
causing high frequency ringing at edges (stepped transition)
- 2) Gaussian → Smooth blurring
- 2) Low pass filter will pass small intensity values
- 3) Median filter removes outliers



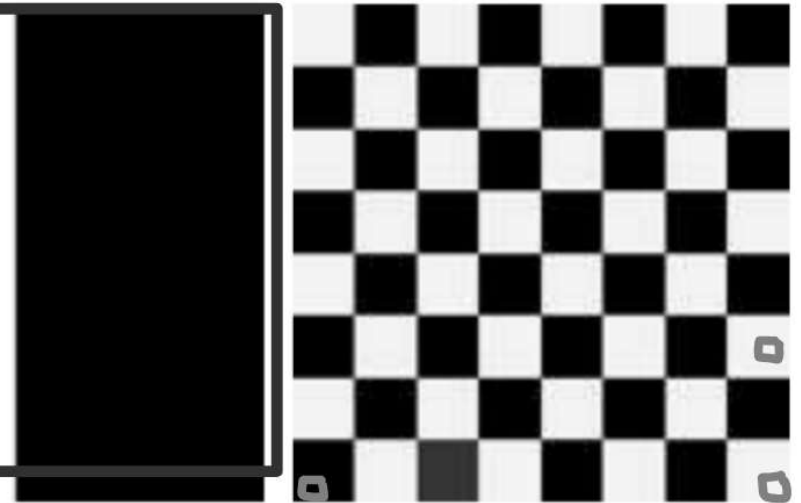
Same Histogram, Different Images — After Blurring (Problem 20)

Problem 20 (G&W 3.14)

The two **binary** images shown are quite different, but share the **same histogram**.

Each image is blurred with a **3×3 averaging kernel**.

- Would the histograms of the blurred images **still be equal**? Explain.
- If **no**, sketch the two resulting histograms.



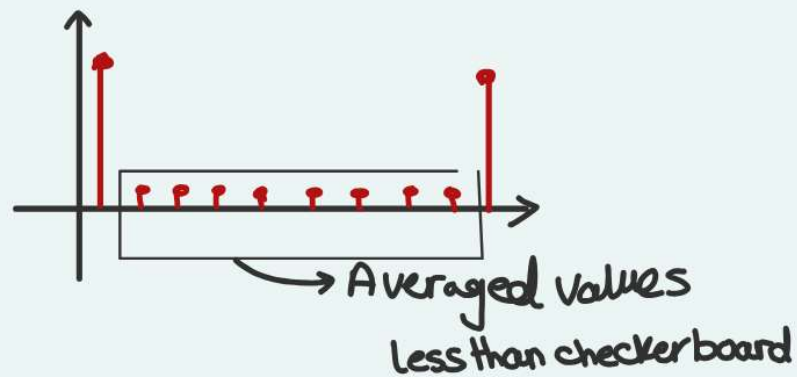
Left: half-black image; Right: checkerboard image



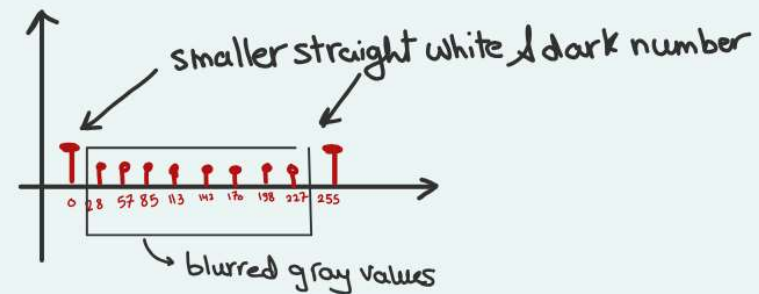
Solution — Problem 20

Problem 20 Solution

a) No, blurred histograms aren't equal
Half black preserves white & black pixels
except at boundary between



$\frac{9}{9} \times 255$ in white, $\frac{9}{9} \times 0$ in black, edges have mix



1	28
2	57
3	85
4	113
5	142
6	170
7	198
8	227

checkerboard Average of 0 & 255 or portions
each time depending on kernel size
for 3×3 kernel
using statistical properties
& all values have equal probabilities (assumption)

Good Luck!

Ain Shams University

ECE 359 — Digital Image Processing & Multimedia
